



340254

20.5, 필기 25.5, 24.7, 24.2, 22.7, 20.6



131 애플리케이션 테스트의 기본 원리

(A)

- 파레토 법칙(Pareto Principle) : 애플리케이션의 20%에 해당하는 코드에서 전체 결함의 80%가 발견된다는 법칙
- 살충제 패러독스(Pesticide Paradox) : 동일한 테스트 케이스로 동일한 테스트를 반복하면 더 이상 결함이 발견되지 않는 현상
- 오류-부재의 궤변(Absence of Errors Fallacy) : 소프트웨어의 결함을 모두 제거해도 사용자의 요구사항을 만족시키지 못하면 해당 소프트웨어는 품질이 높다고 말할 수 없는 것

340255

20.7



132 프로그램 실행 여부에 따른 테스트

(A)

20.7

정적 테스트

- 프로그램을 실행하지 않고 명세서나 소스 코드를 대상으로 분석하는 테스트이다.
- 소스 코드에 대한 코딩 표준, 코딩 스타일, 코드 복잡도, 남은 결함 등을 발견하기 위해 사용한다.
- 종류 : 워크스루, 인스펙션, 코드 검사 등

동적 테스트

- 프로그램을 실행하여 오류를 찾는 테스트이다.
- 소프트웨어 개발의 모든 단계에서 테스트를 수행한다.
- 종류 : 블랙박스 테스트, 화이트박스 테스트

340260

필기 24.2, 23.7, 22.3, 21.5, 20.8, 20.6



133 화이트박스 테스트의 종류

(B)

필기 24.2, 23.7, 22.3, 21.5, 20.8, 20.6

기초 경로 검사(Base Path Testing)

- 테스트 케이스 설계자가 절차적 설계의 논리적 복잡성을 측정할 수 있게 해주는 테스트 기법이다.
- 대표적인 화이트박스 테스트 기법이다.

필기 22.3

제어 구조 검사(Control Structure Testing)

- 조건 검사(Condition Testing) : 프로그램 모듈 내에 있는 논리적 조건을 테스트하는 테스트 케이스 설계 기법
- 루프 검사(Loop Testing) : 프로그램의 반복(Loop) 구조에 초점을 맞춰 실시하는 테스트 케이스 설계 기법
- 데이터 흐름 검사(Data Flow Testing) : 프로그램에서 변수의 정의와 변수 사용의 위치에 초점을 맞춰 실시하는 테스트 케이스 설계 기법

340540

25.11, 25.7, 25.4, 24.10, 24.4, 23.7, 23.4, 21.7, 20.10



134 화이트박스 테스트의 검증 기준

(A)

24.10, 21.7

문장 검증 기준(Statement Coverage)

소스 코드의 모든 구문이 한 번 이상 수행되도록 테스트 케이스를 설계한다.

25.7, 25.4, 24.10, 23.4, 21.7, 20.10

결정 검증 기준(Decision Coverage)

- 소스 코드의 모든 조건문에 대해 조건식의 결과가 True인 경우와 False인 경우가 한 번 이상 수행되도록 테스트 케이스를 설계한다.
- 분기 검증 기준(Branch Coverage)이라고도 한다.

25.11, 24.10, 23.7, 21.7

조건 검증 기준(Condition Coverage)

소스 코드의 조건문에 포함된 개별 조건식의 결과가 True인 경우와 False인 경우가 한 번 이상 수행되도록 테스트 케이스를 설계한다.

조건/결정 검증 기준(Condition/Decision Coverage)

결정 검증 기준과 조건 검증 기준을 모두 만족하는 설계로, 조건문이 True인 경우와 False인 경우에 따라 조건 검증 기준의 입력 데이터를 구분하는 테스트 케이스를 설계한다.

변경 조건/결정 검증 기준(Modified Condition/Decision Coverage)

조건/결정 검증 기준을 향상시킨 검증 기준으로, 개별 조건식이 다른 개별 조건식의 영향을 받지 않고 전체 조건식의 결과에 독립적으로 영향을 주도록 테스트 케이스를 설계한다.

다중 조건 검증 기준(Multiple Condition Coverage)

소스 코드의 조건문에 포함된 모든 개별 조건식의 모든 조합을 고려하도록 테스트 케이스를 설계한다.

340262



135 블랙박스 테스트

20.10



- 블랙박스 테스트(Black Box Test)는 소프트웨어가 수행할 특정 기능을 알기 위해서 각 기능이 완전히 작동되는 것을 입증하는 테스트로, 기능 테스트라고도 한다.
- 사용자의 요구사항 명세를 보면서 테스트한다.
- 주로 구현된 기능을 테스트한다.
- 소프트웨어 인터페이스를 통해 실시된다.

340263



23.10, 22.10, 22.5, 21.10, 21.4, 20.11, 필기 25.8, 25.5, 25.2, 24.7, 24.5, 23.5, ...

136 블랙박스 테스트의 종류



23.10, 22.5, 21.4, 20.11, 필기 25.8, 25.5, 24.7, 24.5, 23.5, 20.9, 20.8

동치 분할 검사(Equivalence Partitioning Testing)

- 프로그램의 입력 조건에 타당한 입력 자료와 타당하지 않은 입력 자료의 개수를 균등하게 하여 테스트 케이스를 정하고, 해당 입력 자료에 맞는 결과가 출력되는지 확인하는 기법이다.
- 동등 분할 기법 또는 동치 클래스 분해라고도 한다.

22.10, 22.5, 21.4, 필기 25.8, 25.5, 25.2, 24.7, 21.3, 20.9, 20.8, 20.6

경계값 분석(Boundary Value Analysis)

입력 조건의 중간값보다 경계값에서 오류가 발생할 확률이 높다는 점을 이용하여 입력 조건의 경계값을 테스트 케이스로 선정하여 검사하는 기법이다.

22.5, 21.10, 필기 20.9

원인-효과 그래프 검사(Cause-Effect Graphing Testing)

입력 데이터 간의 관계와 출력에 영향을 미치는 상황을 체계적으로 분석한 다음 효용성이 높은 테스트 케이스를 선정하여 검사하는 기법이다.

필기 25.5, 24.7, 20.8

오류 예측 검사(Error Guessing)

과거의 경험이나 확인자의 감각으로 테스트하는 기법이다.

비교 검사(Comparison Testing)

여러 버전의 프로그램에 동일한 테스트 자료를 제공하여 동일한 결과가 출력되는지 테스트하는 기법이다.

예제 A 애플리케이션에서 평가점수에 따른 성적부여 기준이 다음과 같을 때, 동치 분할 검사와 경계값 분석의 테스트 케이스를 확인시오.

평가점수	성적
90~100	A
80~89	B
70~79	C
0~69	D

〈동치 분할 검사〉

테스트 케이스	1	2	3	4
입력값	60	75	82	96
예상 결과값	D	C	B	A
실제 결과값	D	C	B	A

해설 동치 분할 검사는 입력 자료에 초점을 맞춰 테스트 케이스를 만들어 검사하므로 평가점수를 입력한 후 점수에 맞는 성적이 출력되는지 확인한다.

〈경계값 분석〉

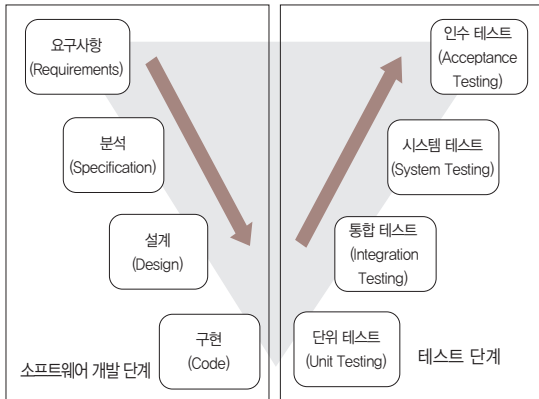
테스트 케이스	1	2	3	4	5	6	7	8	9	10
입력값	-1	0	69	70	79	80	89	90	100	101
예상 결과값	오류	D	D	C	C	B	B	A	A	오류
실제 결과값	오류	D	D	C	C	B	B	A	A	오류

해설 경계값 분석은 입력 조건의 경계값을 테스트 케이스로 선정하여 검사하므로 평가점수의 경계값에 해당하는 점수를 입력한 후 올바른 성적이 출력되는지 확인한다.



137 개발 단계에 따른 애플리케이션 테스트 (A)

- 소프트웨어의 개발 단계에 따라 단위 테스트, 통합 테스트, 시스템 테스트, 인수 테스트로 분류된다. 이렇게 분류된 것을 테스트 레벨이라고 한다.
- 애플리케이션 테스트와 소프트웨어 개발 단계를 연결하여 표현한 것을 V-모델이라고 한다.



소프트웨어 생명 주기의 V-모델



138 단위 테스트 (A)

- 단위 테스트(Unit Test)는 코딩 직후 소프트웨어 설계의 최소 단위인 모듈이나 컴포넌트에 초점을 맞춰 테스트하는 것이다.
- 인터페이스, 외부적 I/O, 자료 구조, 독립적 기초 경로, 오류 처리 경로, 경계 조건 등을 검사한다.
- 사용자의 요구사항을 기반으로 한 기능성 테스트를 최우선으로 수행한다.
- 구조 기반 테스트와 명세 기반 테스트로 나뉘지만 주로 구조 기반 테스트를 시행한다.



139 통합 테스트 (A)

- 통합 테스트(Integration Test)는 단위 테스트가 완료된 모듈들을 결합하여 하나의 시스템으로 완성시키는 과정에서의 테스트를 의미한다.
- 모듈 간 또는 통합된 컴포넌트 간의 상호 작용 오류를 검사한다.
- 비점진적 통합 방식
 - 단계적으로 통합하는 절차 없이 모든 모듈이 미리 결합되어 있는 프로그램 전체를 테스트하는 방법
 - 종류: 빅뱅 통합 테스트 방식
- 점진적 통합 방식
 - 모듈 단위로 단계적으로 통합하면서 테스트하는 방법
 - 종류: 하향식 통합 테스트, 상향식 통합 테스트, 혼합식 통합 테스트



140 인수 테스트 (A)

- 인수 테스트(Acceptance Test)는 개발한 소프트웨어가 사용자의 요구사항을 충족하는지에 중점을 두고 테스트하는 것이다.
- 인수 테스트는 개발한 소프트웨어를 사용자가 직접 테스트한다.
- 인수 테스트 종류

22.7, 필기 23.7, ... 알파 테스트

- 개발자의 장소에서 사용자가 개발자 앞에서 행하는 테스트 기법
- 테스트는 통제된 환경에서 행해지며, 오류와 사용상의 문제점을 사용자와 개발자가 함께 확인하면서 기록함

22.7, 필기 22.3, ... 베타 테스트

- 선정된 최종 사용자가 여러 명의 사용자 앞에서 행하는 테스트 기법
- 실업무를 가지고 사용자가 직접 테스트

**141 하향식 통합 테스트****(A)**

- 하향식 통합 테스트(Top Down Integration Test)는 프로그램의 상위 모듈에서 하위 모듈 방향으로 통합하면서 테스트하는 기법이다.
- 하향식 통합 테스트 절차
 - ① 주요 제어 모듈은 작성된 프로그램을 사용하고, 주요 제어 모듈의 종속 모듈들은 스텝(Stub)으로 대체한다.
 - ② 깊이 우선 또는 넓이 우선 등의 통합 방식에 따라 하위 모듈인 스텝들이 한 번에 하나씩 실제 모듈로 교체된다.
 - ③ 모듈이 통합될 때마다 테스트를 실시한다.
 - ④ 새로운 오류가 발생하지 않음을 보증하기 위해 회귀 테스트를 실시한다.

※ 스텝(Stub) : 제어 모듈이 호출하는 타 모듈의 기능을 단순히 수행하는 도구로, 일시적으로 필요한 조건만을 가지고 있는 시험용 모듈임

**142 상향식 통합 테스트****(A)**

- 상향식 통합 테스트(Bottom Up Integration Test)는 프로그램의 하위 모듈에서 상위 모듈 방향으로 통합하면서 테스트하는 기법이다.
- 상향식 통합 테스트 절차
 - ① 하위 모듈들을 클러스터(Cluster)로 결합한다.
 - ② 상위 모듈에서 데이터의 입·출력을 확인하기 위해 더미 모듈인 드라이버(Driver)를 작성한다.
 - ③ 통합된 클러스터 단위로 테스트한다.
 - ④ 테스트가 완료되면 클러스터는 프로그램 구조의 상위로 이동하여 결합하고 드라이버는 실제 모듈로 대체된다.

※ 테스트 드라이버(Test Driver) : 테스트 대상의 하위 모듈을 호출하고, 파라미터를 전달하고, 모듈 테스트 수행 후의 결과를 도출하는 도구임

**143 회귀 테스트****(A)**

- 회귀 테스트(Regression Test)는 통합 테스트로 인해 변경된 모듈이나 컴포넌트에 새로운 오류가 있는지 확인하는 테스트이다.
- 이미 테스트된 프로그램의 테스트를 반복하는 것이다.
- 회귀 테스트는 수정한 모듈이나 컴포넌트가 다른 부분에 영향을 미치는지, 오류가 생기지 않았는지 테스트하여 새로운 오류가 발생하지 않음을 보증하기 위해 반복 테스트한다.

**144 테스트 오라클****(C)**

- 테스트 오라클(Test Oracle)은 테스트 결과가 올바른지 판단하기 위해 사전에 정의된 참값을 대입하여 비교하는 기법 및 활동을 말한다.
- 결과를 판단하기 위해 테스트 케이스에 대한 예상 결과를 계산하거나 확인한다.
- 테스트 오라클의 특징 : 제한된 검증, 수학적 기법, 자동화 기능

**145 테스트 오라클의 종류****(A)**

- 참(True) 오라클 : 모든 테스트 케이스의 입력 값에 대해 기대하는 결과를 제공하는 오라클로, 발생된 모든 오류를 검출할 수 있음
- 샘플링(Sampling) 오라클 : 특정한 몇몇 테스트 케이스의 입력 값들에 대해서만 기대하는 결과를 제공하는 오라클로 전수 테스트가 불가능한 경우 사용하는 오라클
- 추정(Heuristic) 오라클 : 특정 테스트 케이스의 입력 값에 대해 기대하는 결과를 제공하고, 나머지 입력 값들에 대해서는 추정으로 처리하는 오라클
- 일관성 검사(Consistent) 오라클 : 애플리케이션에 변경이 있을 때, 테스트 케이스의 수행 전과 후의 결과 값이 동일한지를 확인하는 오라클



146 애플리케이션 성능 측정 지표 (A)

- 처리량(Throughput) : 일정 시간 내에 애플리케이션이 처리하는 일의 양
- 응답 시간(Response Time) : 애플리케이션에 요청을 전달한 시간부터 응답이 도착할 때까지 걸린 시간
- 경과 시간(Turn Around Time) : 애플리케이션에 작업을 의뢰한 시간부터 처리가 완료될 때까지 걸린 시간
- 자원 사용률(Resource Usage) : 애플리케이션이 의뢰한 작업을 처리하는 동안의 CPU 사용량, 메모리 사용량, 네트워크 사용량 등 자원 사용률



147 빅오 표기법으로 표현한 최악의 시간 복잡도 (B)

필기 23.7, 22.7, ... $O(1)$	입력값(n)에 관계 없이 일정하게 문제 해결에 하나의 단계만을 거침 예) 스택의 삽입(Push), 삭제(Pop)
$O(\log_2 n)$	문제 해결에 필요한 단계가 입력값(n) 또는 조건에 의해 감소함 예) 이진 트리(Binary Tree), 이진 검색(Binary Search)
$O(n)$	문제 해결에 필요한 단계가 입력값(n)과 1:1의 관계를 가짐 예) for문
필기 25.5, 24.7, ... $O(n \log_2 n)$	문제 해결에 필요한 단계가 $n(\log_2 n)$ 번만큼 수행됨 예) 힙 정렬(Heap Sort), 2-Way 합병 정렬(Merge Sort)
$O(n^2)$	문제 해결에 필요한 단계가 입력값(n)의 제곱만큼 수행됨 예) 삽입 정렬(Insertion Sort), 쉘 정렬(Shell Sort), 선택 정렬(Selection Sort), 버블 정렬(Bubble Sort), 퀵 정렬(Quick Sort)
$O(2^n)$	문제 해결에 필요한 단계가 2의 입력값(n) 제곱만큼 수행됨 예) 피보나치 수열(Fibonacci Sequence)



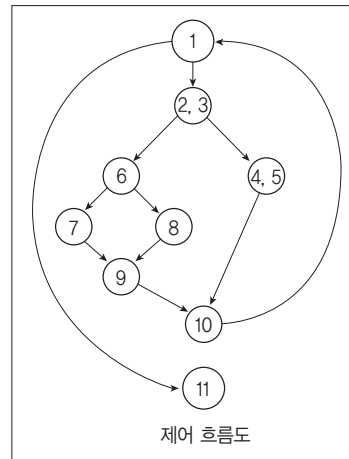
148 순환 복잡도 (C)

- 순환 복잡도(Cyclomatic Complexity)는 한 프로그램의 논리적인 복잡도를 측정하기 위한 소프트웨어의 척도이다.
- 맥케이크 순환도(McCabe's Cyclomatic)라고도 한다.
- 제어 흐름도 G에서 순환 복잡도 $V(G)$ 는 다음과 같은 방법으로 계산할 수 있다.

방법 1 순환 복잡도는 제어 흐름도의 영역 수와 일치하므로 영역 수를 계산한다.

방법 2 $V(G) = E - N + 2$: E는 화살표 수, N은 노드의 수

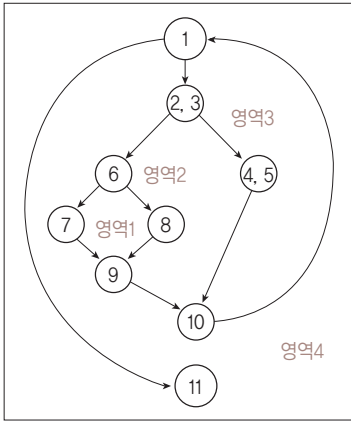
예제 제어 흐름도가 다음과 같을 때 순환 복잡도(Cyclomatic Complexity)를 계산하시오.



해설

순환 복잡도는 다음 두 가지 방법으로 구할 수 있습니다.

방법 1 제어 흐름도에서 화살표로 구분되는 각 영역의 개수를 구하면 4입니다.



방법 2 순환 복잡도 = 화살표의 수 - 노드의 수 + 2
이므로 $11 - 9 + 2 = 4$ 입니다.



149 소스 코드 최적화



- 소스 코드 최적화는 나쁜 코드(Bad Code)를 배제하고, 클린 코드(Clean Code)로 작성하는 것이다.
- 나쁜 코드로 작성된 애플리케이션의 코드를 클린 코드로 수정하면 애플리케이션의 성능이 개선된다.
- 클린 코드(Clean Code) : 누구나 쉽게 이해하고 수정 및 추가할 수 있는 단순, 명료한 코드, 즉 잘 작성된 코드
- 나쁜 코드(Bad Code)
 - 프로그램의 로직(Logic)이 복잡하고 이해하기 어려운 코드
 - 스파게티 코드(Spaghetti Code) : 코드의 로직이 서로 복잡하게 얽혀 있는 코드
 - 외계인 코드(Alien Code) : 아주 오래되거나 참고문서 또는 개발자가 없어 유지보수 작업이 어려운 코드

340290



150 클린 코드 작성 원칙



필기 24.2, 22.7, 22.3, 20.9, 20.8

필기 24.2, 22.7, 22.3

가독성

- 누구든지 코드를 쉽게 읽을 수 있도록 작성한다.
- 코드 작성 시 이해하기 쉬운 용어를 사용하거나 들여쓰기 기능 등을 사용한다.

필기 24.2, 22.7, 20.9, 20.8

단순성

- 코드를 간단하게 작성한다.
- 한 번에 한 가지를 처리하도록 코드를 작성하고 클래스/메소드/함수 등을 최소 단위로 분리한다.

필기 24.2, 22.7, 22.3, 20.8

의존성 배제

- 코드가 다른 모듈에 미치는 영향을 최소화한다.
- 코드 변경 시 다른 부분에 영향이 없도록 작성한다.

필기 24.2, 22.7, 22.3, 20.8

중복성 최소화

- 코드의 중복을 최소화한다.
- 중복된 코드는 삭제하고 공통된 코드를 사용한다.

필기 22.3

추상화

상위 클래스/메소드/함수에서는 간략하게 애플리케이션의 특성을 나타내고, 상세 내용은 하위 클래스/메소드/함수에서 구현한다.



151 소스 코드 품질 분석 도구



소스 코드 품질 분석 도구는 소스 코드의 코딩 스타일, 코드에 설정된 코딩 표준, 코드의 복잡도, 코드에 존재하는 메모리 누수 현상, 스레드 결함 등을 발견하기 위해 사용하는 분석 도구이다.

22.5, 필기 25.5, 23.7, 21.8, 20.9, 20.6

정적 분석(Static Analysis) 도구

- 작성한 소스 코드를 실행하지 않고 코딩 표준이나 코딩 스타일, 결함 등을 확인하는 코드 분석 도구이다.
- 종류 : pmd, cppcheck, SonarQube, checkstyle, ccm, cobertura 등

22.5

동적 분석(Dynamic Analysis) 도구

- 작성한 소스 코드를 실행하여 코드에 존재하는 메모리 누수, 스레드 결함 등을 분석하는 도구이다.
- 종류 : Avalanche, Valgrind 등